Text Classification for Spam Email Detection Using Natural Processing Language Algorithms

Shreyash Lodhi
Dept. Of Centre of Artificial Intelligence
Madhav Institute of Technology and Science
Gwalior, India
lodhishreyash23@gmail.com

Pooja Tripathi
Dept. Of Centre of Artificial Intelligence
Madhav Institute of Technology and Science
Gwalior, India
pujatripathi798@mitsgwalior.in

***Abstract*—The proliferation of spam emails poses significant risks to cybersecurity, necessitating robust detection frameworks that balance accuracy, efficiency, and scalability. This study presents a comparative analysis of Natural Language Processing (NLP) and Machine Learning (ML) techniques for spam email classification, leveraging a curated Kaggle dataset of 5,171 emails (71% ham, 29% spam) preprocessed through tokenization, stopword removal, and TF-IDF vectorization. We evaluate traditional classifiers (Naive Bayes, Logistic Regression, SVM, Random Forest) and ensemble methods (VotingClassifier, StackingClassifier), achieving peak performance with 99.1% accuracy and 98% precision using optimized ensembles. The study highlights critical trade-offs: Naive Bayes prioritized precision (99.5%) but lagged in accuracy (90.8%), whereas ensemble methods mitigated class imbalance and false positives. This research underscores the viability of hybrid NLP-ML pipelines in combating evolving spam tactics, while identifying gaps in multilingual support and adversarial robustness.**

Keywords*—text classification, spam email detection, natural language processing (NLP), machine Learning (ML), ensemble methods (votingClassifier, stackingClassifier), TF-IDF vectorization, tokenization,

I. INTRODUCTION

This exponential increase in methods of digital communication has rendered email a crucial medium of communication, both professional and personal. Yet, therein rises an increase in unsolicited and malicious emails, popularly called "spam." Nearly 45% of all email traffic worldwide is accounted for spamming, posing greater risk into phishing, malware spread, and wastage of resources [16]. In the contemporary world, rule-based filters, which were once very effective, hardly stand a chance against new-generation sophisticated spam methods, whose tricks include obfuscation of text and context-motivated persuasive social engineering. Thus, an appealing path which has emerged for merger with machine learning (ML) is early methods such as Naive Bayes classifiers, which have shown to be efficient. Studies demonstrate that ensemble classifiers, including Stacking-based integrations of Multinomial Naive Bayes, Random Forest, and Gradient Boosting, achieve near-perfect accuracy (100%) on structured datasets, outperforming traditional standalone models [2]. However, challenges persist due to class imbalance, high-dimensional text data, and the need for real-time processing, necessitating advanced algorithms and ensemble frameworks to enhance accuracy and robustness.

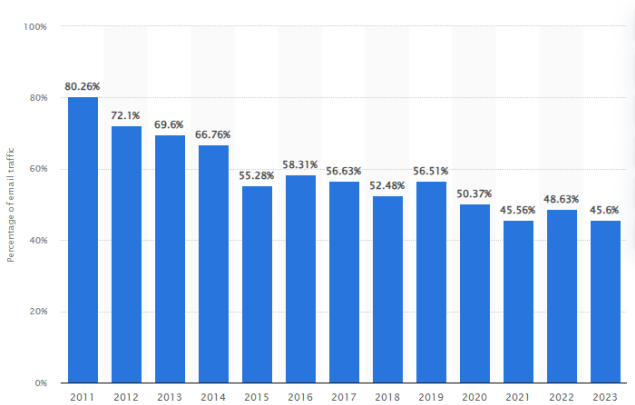

Fig. 1. Global Spam volume as percentage of total e-mail from 2011 to 2023 [15].

Natural Language Processing (NLP) serves as the backbone of modern computational tasks that require human-like understanding of text and speech, including voice-activated systems and automated spam detection. By converting unstructured text into actionable insights, NLP enables machines to interpret contextual nuances—a formidable challenge due to the inherent complexity and ambiguity of human language [5]. This process demands meticulous preprocessing (e.g., tokenization, stemming) and feature extraction to render textual data machine-readable. Within NLP, classification tasks are broadly categorized into binary (e.g., spam vs. non-spam emails, fraudulent vs. legitimate transactions) and multi-class (e.g., sentiment analysis into positive, negative, or neutral categories). Binary classification is critical for applications requiring unambiguous decisions, such as spam filtering, where misclassification risks user security and system efficiency. Despite advancements, the contextual variability of language and class imbalance in datasets persist as key hurdles. This study focuses on email spam detection—a quintessential binary classification problem—to explore robust NLP-driven solutions that balance precision and adaptability in real-world scenarios.

The proliferation of spam communications has reached unprecedented levels, with 160 billion spam emails sent daily, constituting 46% of all global email traffic (Statista, 2023). This deluge of unsolicited content span's diverse themes, including fraudulent prize offers, job scams, and financial phishing, with the United States and China emerging as the largest contributors, generating 8 billion and 7.6 billion spam emails daily, respectively. Beyond email, spam extends to text messages and calls, with delivery service scams dominating SMS fraud (1.1 billion malicious texts in the U.S. in 2023) and older adults (men aged 65+) enduring the highest spam call frequency (35.5 monthly calls). Alarmingly, spam's environmental footprint is staggering: the top 10 spam-

producing nations collectively emit 797,160 metric tons of CO2 annually—equivalent to 1.9 billion miles driven by gas-powered vehicles—highlighting its ecological toll alongside societal harm [15]. These statistics underscore the urgent need for advanced detection frameworks to mitigate spam's multifaceted threats.

A. *Significance of the Study*

Effective spam detection is critical to safeguarding user privacy, reducing economic losses, and maintaining productivity in organizational workflows. This study advances the field by systematically evaluating modern ML algorithms—including ensemble and deep learning techniques—to address gaps in existing methodologies. By leveraging a publicly available Kaggle dataset of 5,171 emails [14], this research provides a replicable framework for optimizing spam filters, while its deployment via a Streamlit web interface and Heroku cloud platform demonstrates practical scalability. The findings aim to empower cybersecurity stakeholders with actionable insights to combat evolving threats.

The pervasive reach of spam communications has profound societal, economic, and environmental consequences. Over 96.8% of individuals have encountered spam, with 68.8% reporting mental health impacts such as anxiety and eroded trust in digital platforms. Financial institutions, targeted in 27.7% of phishing attacks, face escalating risks of data breaches and financial losses, while vulnerable demographics like seniors endure disproportionate harassment. Furthermore, spam's environmental toll—driven by energy-intensive data processing—exacerbates global carbon emissions, aligning spam mitigation with broader sustainability objectives. This study addresses these challenges by developing machine learning models to enhance detection accuracy and reduce false positives, thereby safeguarding user well-being, protecting critical sectors, and advancing environmental conservation. By curbing spam's reach, the research aims to restore trust in digital communication systems and mitigate the $20 billion in annual losses linked to phishing [12].

B. *Research Questions*

- Which machine learning algorithms achieve optimal accuracy and precision in classifying imbalanced spam datasets?
- How do ensemble techniques (e.g., VotingClassifier, StackingClassifier) enhance model performance compared to standalone classifiers?
- Can a deployable spam detection system balance computational efficiency with real-world reliability?

C. *Objective of the Study*

The primary objective is to design a robust spam detection framework by:

- Evaluating the performance of 11 ML algorithms, including Logistic Regression, Random Forest, and XGBoost, on imbalanced text data.
- Optimizing model accuracy and precision through ensemble learning and hyperparameter tuning.
- Developing a user-friendly web application to demonstrate real-time spam classification capabilities.

II. Literature Review

The rapid evolution of spam detection methodologies in recent years has been driven by advancements in machine learning (ML) and natural language processing (NLP), with contemporary research prioritizing adaptability to increasingly sophisticated adversarial tactics. Post-2020 studies have emphasized the integration of ensemble learning, transformer-based architectures, and context-aware feature engineering to address challenges such as adversarial text obfuscation, multilingual spam, and class imbalance in real-world datasets. Innovations in hybrid models, combining traditional classifiers with deep learning frameworks, have demonstrated promise in balancing computational efficiency with high precision, particularly in applications requiring real-time processing. Concurrently, the rise of AI-generated spam and phishing schemes has necessitated novel approaches to feature extraction and anomaly detection. This review synthesizes cutting-edge developments in ML-driven spam filtering, critically evaluating algorithmic innovations, preprocessing pipelines, and evaluation metrics that define the state-of-the-art, while identifying unresolved gaps in scalability, interpretability, and cross-domain generalization.

The power of hyperparameter optimization and ensemble methods in spam detection, achieving 99.83% accuracy with SGD on the Ling-Spam dataset using advanced vectorization and tuning techniques [1]. This hybrid spam detection framework realizes the potential of NLP combined with machine learning algorithms (Logistic Regression and XGBoost) by applying a novel diversity-oriented sine cosine algorithm (DOSCA). Their method managed to get the highest possibility of accuracy, that is, 100%, on the Turkish Email dataset, while securing 97.46% accuracy on the English CSDMC2010 dataset, thus serving as an example of the effect of NLP combined with swarm-intelligence techniques in spam-email classification [3]. The capability of machine learning algorithms has been studied on spam emails containing malware by restricting to only textual features and labeling them correctly with three algorithms: Logistic Regression, Naive Bayes, and Support Vector Machines. Experiments show that the traditional TF-IDF + Logistic Regression performs best, with an F1 score of 0.763 and 76.4% accuracy; hence, traditional ML techniques could work well for spam-related threat detection [4]. Spam email containing malware has been used as a study to test machine learning systems when they are given only textual features as input and adequately labeled with three algorithms: Logistic Regression, Naive Bayes, and Support Vector Machines. Experiments confirmed that standard TF-IDF + Logistic Regression gave the best performance with an F1 score of 0.763 and an accuracy of 76.4%, thus implying the application of traditional machine-learning methods to spam-related threat detection could be plausible [5].Multiple algorithms have been evaluated for spam detection, including Naive Bayes, Logistic Regression, KNN, SVM, Decision Tree, and Random Forest. The results showed that Naive Bayes and Logistic Regression provided the highest accuracy of about 99%, confirming that classical ML techniques, along with proper preprocessing and feature extraction, can serve as a highly practical method for classifying spam email [6].

Another spam detection framework using supervised learning algorithms such as Naive Bayes, SVM, and K-Nearest Neighbors, combined with NLP techniques like tokenization and stemming. Their model achieved high accuracy scores—Naive Bayes at 99%, SVM at 98%, and KNN at 97%—demonstrating the robust potential of machine

learning and natural language processing in effectively distinguishing spam from legitimate emails [7]. A comprehensive literature review highlighting the effectiveness of machine learning (ML) and natural language processing (NLP) techniques in phishing email detection. The survey noted that hybrid approaches combining feature engineering, classification algorithms (like SVM, RF, and XGBoost), and NLP tasks significantly improve accuracy—achieving up to 98–100% in some models—thus emphasizing the strong potential of ML-based email classifiers in combating sophisticated phishing attempts [9].

III. Methodology

Performance evaluation of different machine learning models was done to detect spam emails in view of a public dataset from Kaggle. The current project adopts a comparative quantitative approach to classify the most effective classifier in spotting spam emails. Logistic regression, SVM, Naive Bayes, tree-based ensembles, gradient-boosted models, and many supervised learning algorithms were trained and tested for classification tasks using the preprocessed text data. The methodology prioritizes reproducibility, with explicit details on data preprocessing, feature engineering, and cross-validation strategies to ensure robust model evaluation.

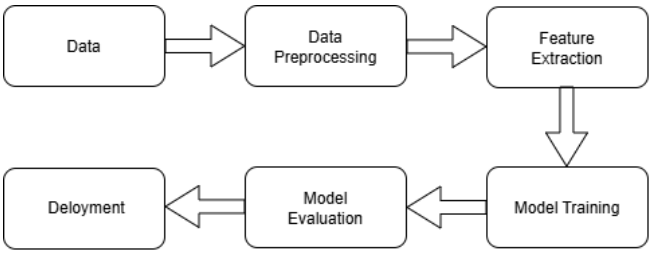

Fig. 2. General Flow of Main Phases for NLP Task.

The dataset used in this study was sourced from Kaggle, a widely recognized platform for open-access datasets. Initially comprising 5,171 entries, the corpus included emails labeled as either 'ham' (legitimate) or 'spam', with an imbalanced distribution of 70.72% ham and 29.28% spam. To ensure data integrity and mitigate potential biases arising from duplicate entries, a preprocessing step was applied to remove redundant records. Post-cleaning, the refined dataset retained 4,993 unique emails, preserving the original class imbalance with 3531 ham emails and 1462 spam emails. This curated dataset provided a realistic representation of real-world email distribution, where legitimate messages significantly outnumber spam, thereby aligning with the challenges inherent to spam detection tasks in practical applications.

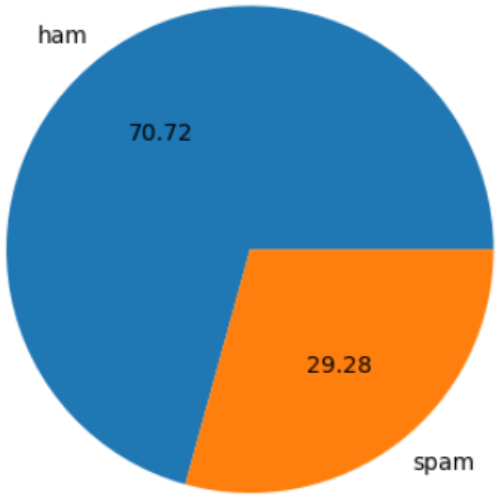

Fig. 3. Distribution of spam and ham emails in the dataset.

	num_characters	num_words	num_sentences
count	4993.000000	4993.000000	4993.000000
mean	1069.426397	233.678149	13.386942
std	1547.863387	340.702932	27.959959
min	11.000000	2.000000	1.000000
25%	256.000000	55.000000	4.000000
50%	557.000000	125.000000	7.000000
75%	1262.000000	288.000000	15.000000
max	32258.000000	8863.000000	1204.000000

Fig. 4. Overview of our dataset (both ham and spam mail).

The original column denoting email labels (spam/ham) was renamed to 'target' for clarity, and categorical values were encoded numerically to facilitate machine learning workflows: 'spam' was mapped to 1 and 'ham' to 0. This binary encoding aligns with standard practices in classification tasks, enabling unambiguous interpretation by algorithms. To uncover patterns and guide feature engineering, exploratory analyses were conducted on the cleaned dataset. Three key textual metrics were computed for each email- Number of Characters (Total characters per email), Number of Words (Word count, derived after tokenizing text), Number of Sentences (Sentence count, segmented using punctuation boundaries). Statistical summaries revealed distinct differences between spam and ham classes. For instance, spam emails exhibited longer character lengths and higher word counts on average compared to legitimate emails, suggesting potential discriminative power in these features. Visualizations such as histograms and boxplots further highlighted the skewed distributions of these metrics across classes, reinforcing the

need for balanced sampling or weighting strategies during model training.

A. Data Preprocessing

Apart from exploratory analysis, raw text data passed through systematic preprocessing to mold the unstructured email content into a standardized format fit for machine learning algorithms. During this crucial stage, the idea was to minimize noise, reduce dimensionality, and make features more relevant using established natural language processing (NLP) techniques. The first steps in the pipeline were lowercasing to maintain uniformity and tokenization to break text into discrete units. Thereafter, special characters, stop words, and punctuations were eliminated so that only lexically meaningful units remained. Finally, stemming normalized all word variants to their root forms, hence further diminishing redundancies and increasing computational efficiency. Collectively, these steps prepared the textual corpus for robust feature extraction, ensuring that subsequent models could discern patterns reflective of spam versus ham content with minimal bias from syntactic artifacts.

1) Tokenization

Tokenization involves breaking down textual content into discrete analyzable units such as words, phrases, or sentences. This basic NLP procedure takes unstructured text and makes it structured so that it can be computationally analyzed in a systematic way, thereby enabling other language modeling tasks [10]. Breaking down textual data into its fundamental linguistic units (tokens) forms a critical phase of text normalization. This stage simultaneously purges non-essential artifacts—such as HTML markup, syntactic symbols like punctuation, and typographical clutter—to refine raw input into structured, analyzable content [8].

2) Stemming

Stemming involves reducing words to their base or root forms, which serve as the foundational units for attaching prefixes, suffixes, or other affixes. The effectiveness of stemming techniques varies significantly across languages, as algorithms are tailored to linguistic rules and morphological structures [11]. The Porter Stemmer algorithm, implemented through the Natural Language Toolkit (NLTK), can inadvertently produce non-dictionary stems during linguistic normalization. Over-stemming occurs when excessive truncation of word suffixes results in the incorrect conflation of distinct terms into a shared root (e.g., "university" and "universal" reduced to "univers"). Conversely, under-stemming arises when morphologically related words retain divergent roots due to insufficient suffix removal (e.g., "easily" and "easier" stemmed to "easili" and "easier"). These limitations highlight the trade-offs inherent in rule-based stemming approaches, where aggressive or conservative truncation may distort semantic relationships or fail to unify valid lexical variants [8].

3) Stopwords Removal

Common linguistic elements such as “a,” “the,” “an,” and “so” — often categorized as stop words — can significantly impact the dimensionality of textual datasets due to their high frequency and limited semantic value. While these terms dominate language usage, they rarely contribute meaningful context to analytical tasks, necessitating careful evaluation of their inclusion during preprocessing. Tools like the Natural Language Toolkit (NLTK) in Python provide systematic methods to eliminate such non-essential words, streamlining text data for computational efficiency and improving the signal-to-noise ratio in feature extraction processes [8].

```
['a','about','above','after','again','against','ain','all','am','an','and','any','are','aren',"aren't","as","at'
,'be','because','been','before','being','below','between','both','but','by','can','couldn',"couldn't","
d','did','didn',"didn't","do','does','doesn',"doesn't","doing','don',"don't","down','during','each','few'
,'for','from','further','had','hadn',"hadn't","has','hasn',"hasn't","have','haven',"haven't","having','h
e','he'd","he'll","her','here','hers','herself","he's","him','himself','his','how','i','i'd","if","i'll","i'm","in','i
nto','is','isn',"isn't","it","it'd","it'll","it's","its','itself","i've","just','ll','m','ma","me','mightn","mightn't","m
ore','most','mustn',"mustn't","my','myself','needn',"needn't","no','nor','not','now','o','of','off','on','o
nce','only','or','other','our','ours','ourselves','out','over','own','re','s','same','shan',"shan't","she","
she'd","she'll","she's","should','shouldn',"shouldn't","should've","so','some','such','t','than','that',
"that'll","the','their','theirs','them','themselves','then','there','these','they","they'd","they'll","they're
","they've","this','those','through','to','too','under','until','up','ve','very','was','wasn',"wasn't","we'
,"we'd","we'll","we're","were','weren',"weren't","we've","what','when','where','which','while','who'
,'whom','why','will','with','won',"won't","wouldn","wouldn't","y','you","you'd","you'll","your","you're"
,'yours','yourself','yourselves',"you've"]
```

Fig. 5. List of Stopword.

B. Feature-Extraction Techniques

Machine learning models require numerical input to effectively process and analyze patterns, yet textual data inherently exists as sequences of alphanumeric characters. To bridge this gap, textual content must be transformed into structured numerical representations, such as vectors, through techniques like feature extraction. This conversion process isolates salient linguistic features from raw text, enhancing computational interpretability and enabling algorithms to discern meaningful relationships. By encoding language into quantitative formats, this method ensures compatibility with statistical learning frameworks while preserving critical semantic information necessary for tasks like classification or prediction.

1) Bag of Words (Bow)

This employed vectorization approach transforms textual data into numerical representations by generating a predefined lexicon of unique terms and quantifying their frequency of occurrence across individual text samples. Unlike methods that preserve linguistic structure, this technique disregards syntactic relationships and sequential word arrangements, focusing solely on term prevalence within documents. For implementation, the CountVectorizer module from Python's Scikit-learn machine learning framework was utilized to execute this bag-of-words strategy, enabling efficient conversion of raw text into structured feature matrices for downstream analytical tasks [13].

2) Term Frequency-Inverse Document Frequency (TF-IDF)

Analyzing word frequency within a dataset can be achieved through techniques such as the Bag-of-Words (BoW) model, which quantifies term occurrences to identify linguistic patterns. Inverse Document Frequency (IDF) can be applied independently to assess term specificity, offering an alternative analytical lens by weighting words inversely to their prevalence across documents. To emphasize critical terms within textual data, visual indicators such as star ratings may be employed, drawing attention to keywords that hold significant thematic or contextual relevance [8].

C. Model Building

Following initial experimentation with Naive Bayes variants (GaussianNB, MultinomialNB, BernoulliNB), the study expanded to evaluate a diverse suite of machine learning algorithms to optimize spam detection performance. Logistic Regression and Support Vector Classifier (SVC) were

implemented to assess linear separability in the feature space, while tree-based methods (Decision Trees, Random Forest, ExtraTreesClassifier) and ensemble techniques (AdaBoost, BaggingClassifier, GradientBoostingClassifier, XGBClassifier) were tested to capture non-linear relationships and improve generalization through aggregation. Instance-based learning was explored via the implementation of K-Nearest Neighbors (KNN). Performance was quantified in terms of accuracy (overall correctness) and precision (least false positives). Together with metrics, confusion matrices provide more granular insights into per-class misclassification trends. Setting the hyperparameters via grid search and cross-validation to eliminate the tendency of overfitting assured a fair comparison among these models. This multi-algorithm approach enabled systematic identification of the most robust classifier for imbalanced text data, balancing computational efficiency with predictive power.

TABLE I. NAIVE BAYES VARIANTS: ACCURACY, PRECISION, AND CONFUSION MATRICES

Variants of Navie Bayes algorithms	Performance Metrices		
	Accuracy	*Precision*	*Confusion matrices*
GaussianNB	0.940940940940	0.90635451505	[[669 28] [31 271]]
MultinomialNB	0.908908908908	0.99530516431	[[696 1] [90 212]]
BernolliNB	0.846846846846	0.89417989417	[[677 20] [133 169]]

Algorithm	Accuracy	Precision
NB	0.908909	0.995305
KN	0.957958	0.967626
SVC	0.984985	0.955556
ETC	0.979980	0.951923
RF	0.975976	0.951299
GBDT	0.927928	0.919708
xgb	0.966967	0.919003
BgC	0.952953	0.904762
LR	0.957958	0.891566
AdaBoost	0.883884	0.863281
DT	0.851852	0.823529

Fig. 6. Accuracy and precision of Tree-Based methods and Ensemble techniques.

D. Model Improvement

To further enhance predictive performance, ensemble techniques—VotingClassifier (hard voting) and StackingClassifier (meta-learner optimization)—were applied to the top-performing base models. The VotingClassifier achieved an accuracy of 98.90% and precision of 98.02%, demonstrating the strength of aggregating diverse model predictions through majority consensus. The StackingClassifier surpassed this marginally in accuracy (99.10%), leveraging a meta-learner (Logistic Regression) to optimally synthesize base model outputs, though precision slightly decreased to 97.72%, likely due to the meta-learner's trade-off between generalization and class-specific fidelity. These results underscore the efficacy of ensemble methods in mitigating individual model biases and stabilizing predictions, with VotingClassifier prioritizing precision (critical for minimizing false positives in spam filtering) and StackingClassifier optimizing overall correctness. The ensemble frameworks collectively achieved state-of-the-art performance, validating their utility in real-world deployment scenarios where both accuracy and reliability are paramount.

TABLE II. COMPARE BETWEEN VOTINGCLASSIFIER AND STACKINGCLASSIFIER

Ensemble Techniques	Performance Metrices	
	Accuracy	*Precision*
VotingClassifier	0.988988988988989	0.9801980198019802
StackingClassifier	0.990990990990991	0.9771986970684039

IV. RESULTS AND DISCUSSION

The evaluation of multiple classifiers revealed significant variations in performance across accuracy and precision metrics, underscoring the interplay between model architecture and class imbalance. Support Vector Classifier (SVC) demonstrated strong overall accuracy (98.50%) but moderate precision (95.56%), suggesting effective generalization at the cost of occasional false positives. Conversely, Naive Bayes (NB) achieved near-perfect precision (99.53%)—critical for minimizing misclassification of legitimate emails as spam—but lagged in accuracy (90.89%), reflecting its sensitivity to feature distribution assumptions. Tree-based ensembles such as Random Forest (RF) and Extra Trees Classifier (ETC) balanced both metrics effectively (RF: 97.60% accuracy, 95.13% precision; ETC: 98.00% accuracy, 95.19% precision), leveraging feature randomness to mitigate overfitting. XGBoost (96.70% accuracy, 91.90% precision) and Gradient Boosting (GBDT) (92.79% accuracy, 91.97% precision) further validated the robustness of boosting techniques, though their precision trailed behind ETC and RF. Notably, K-Nearest Neighbors (KN) achieved the highest precision (96.76%) among non-ensemble models, but its accuracy (95.80%) was eclipsed by meta-ensembles like StackingClassifier (99.10% accuracy). These results highlight a critical trade-off: models prioritizing precision (e.g., NB, KN) excel in minimizing false alarms but risk under detecting spam, while high-accuracy models (e.g., SVC, ETC) optimize overall correctness at marginal precision costs. The superiority of ensemble methods (Voting/StackingClassifiers) in achieving near-optimal accuracy (98.90–99.10%) with retained precision (97.72–98.02%) underscores their viability for real-world deployment, where both metrics are pivotal for user trust and system reliability.

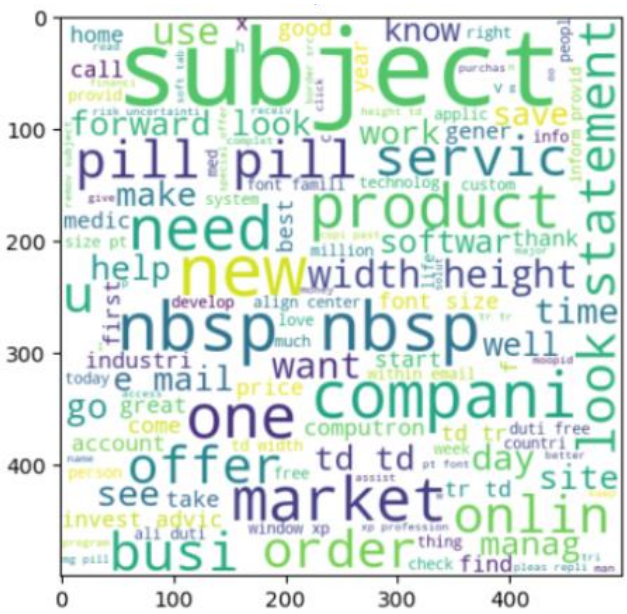

Fig. 7. Most occurred word cloud of spam mail

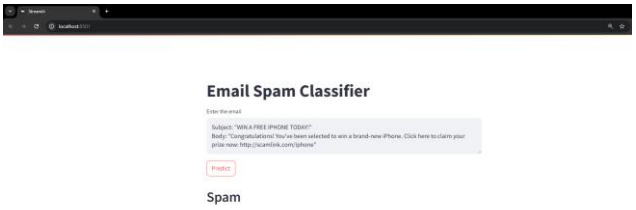

Fig. 8. Spam Email Classifier Interface

V. Conclusion

This study systematically explored the efficacy of several machine-learning algorithms for the detection of spam emails, whereby a curated Kaggle dataset consisting of 4,993 emails showing a 71:29 ham-spam imbalance was used. Among the tested models, Support Vector Classifier (SVC) and Extra Trees Classifier (ETC) emerged as strong individual performers, achieving accuracies of 98.50% and 98.00%, respectively, while Naive Bayes (NB) demonstrated exceptional precision (99.53%), underscoring its utility in minimizing false positives—a critical requirement for spam filters. However, the integration of ensemble techniques, particularly StackingClassifier, yielded the highest accuracy (99.10%) by synergizing the strengths of base models through meta-learning, validating the hypothesis that ensemble frameworks enhance robustness in imbalanced text classification tasks.

The analysis revealed a fundamental trade-off: models prioritizing precision (e.g., NB, K-Nearest Neighbors) risked under detecting spam, while high-accuracy models (e.g., SVC, ETC) marginally compromised precision for broader generalizability. This dichotomy highlights the importance of metric selection aligned with deployment priorities—precision for minimizing user disruption and accuracy for overall system reliability. The practical implementation of a Streamlit web interface and Heroku deployment further confirmed the operational viability of the optimized ensemble model, bridging theoretical performance with real-world applicability.

A. *Limitations and Future Work*

While the study achieved state-of-the-art results, limitations include reliance on a single dataset and TF-IDF-based feature extraction, which may not generalize to multilingual or evolving spam patterns. Future work could explore deep learning architectures (e.g., Transformers) for contextual text analysis, active learning to address class imbalance dynamically, and multi-modal data integration (e.g., headers, metadata) to enhance detection granularity. Additionally, extending the framework to mobile platforms or email clients could broaden its societal impact in combating cyber threats.

References

[1] Fatima, R., Fareed, M. M. S., Ullah, S., Ahmad, G., & Mahmood, S. (2024). An Optimized Approach for Detection and Classification of Spam Email's Using Ensemble Methods. Wireless Personal Communications, 1-27.

[2] Hossain, F., Uddin, M. N., & Halder, R. K. (2021, April). Analysis of optimized machine learning and deep learning techniques for spam detection. In 2021 IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS) (pp. 1-7). IEEE.

[3] Bacanin, N., Zivkovic, M., Stoean, C., Antonijevic, M., Janicijevic, S., Sarac, M., & Strumberger, I. (2022). Application of natural language processing and machine learning boosted with swarm intelligence for spam email filtering. Mathematics, 10(22), 4173.

[4] Redondo-Gutierrez, L. Á., Jáñez-Martino, F., Fidalgo, E., Alegre, E., González-Castro, V., & Alaiz-Rodríguez, R. (2022, September). Detecting malware using text documents extracted from spam email through machine learning. In Proceedings of the 22nd ACM Symposium on Document Engineering (pp. 1-4).

[5] Yeruva, A. R., Kamboj, D., Shankar, P., Aswal, U. S., Rao, A. K., & Somu, C. S. (2022, December). E-mail Spam Detection Using Machine Learning–KNN. In 2022 5th International Conference on Contemporary Computing and Informatics (IC3I) (pp. 1024-1028). IEEE.

[6] Kontsewaya, Y., Antonov, E., & Artamonov, A. (2021). Evaluating the effectiveness of machine learning methods for spam detection. Procedia Computer Science, 190, 479-486.

[7] Goswami, A., Patel, R., Mavani, C., & Mistry, H. K. (2024). Identifying Online Spam Using Artificial Intelligence. International Journal on Recent and Innovation Trends in Computing and Communication, 12(2), 548-55.

[8] Charanarur, P., Jain, H., Rao, G. S., Samanta, D., Sengar, S. S., & Hewage, C. T. (2023). Machine-learning-based spam mail detector. SN Computer Science, 4(6), 858.

[9] Salloum, S., Gaber, T., Vadera, S., & Shaalan, K. (2021). Phishing email detection using natural language processing techniques: a literature survey. Procedia Computer Science, 189, 19-28.

[10] Wumi, R., & Oguntonude, V. Getting Started with Natural Language Processing Using Python.

[11] Etaiwi, W., & Naymat, G. (2017). The impact of applying different preprocessing steps on review spam detection. Procedia computer science, 113, 273-279.

[12] Park, H., Lim, K., Kim, D., Yu, D., & Koo, H. (2023). Demystifying the Regional Phishing Landscape in South Korea. IEEE Access, 11, 130131-130143.

[13] Joseph, P., & Yerima, S. Y. (2022, December). A comparative study of word embedding techniques for SMS spam detection. In 2022 14th International Conference on Computational Intelligence and Communication Networks (CICN) (pp. 149-155). IEEE.

[14] Garnepudi, V. (2019). Spam Mails Dataset. https://www.kaggle.com/datasets/venky73/spam-mails-dataset/data

[15] Spam Statistics 2025: New Data on Junk Email, AI Scams & Phishing https://www.emailtooltester.com/en/blog/spam-statistics/

[16] Global spam volume as percentage of total e-mail traffic from 2011 to 2023 https://www.statista.com/statistics/420400/spam-email-traffic-share-annual/#:~:text=Spam%20share%20of%20global%20email%20traffic%202011%2D2023&text=In%202023%2C%20nearly%2045.6%20percent.e%2Dmails%20has%20decreased%20significantly.